

A Framework for Automated Performance Testing in CI/CD Pipelines: An Experience Report in a Public-Sector Environment

Thiago Silva

Alana Oliveira

Mario Teixeira

Graduate Program in Computer Science

Department of Computer Science

Federal University of Maranhão (UFMA), Brazil

E-mail: thiago.augusto@discente.ufma.br

Abstract

The adoption of Continuous Integration and Continuous Delivery (CI/CD) practices has significantly improved the speed and automation of software delivery. However, while unit and functional tests are commonly integrated into CI/CD pipelines, performance testing is still frequently neglected or executed manually after deployment. This paper presents a framework for integrating automated performance testing into CI/CD pipelines through a parameterizable plug-in capable of generating and executing workloads. The proposed solution enables the continuous validation of non-functional requirements such as response time, scalability, and concurrency limits directly within the delivery pipeline. The framework was evaluated in a real-world public-sector environment based on a microservices architecture deployed on Kubernetes. A factorial experimental design was applied to analyze the effects of concurrency levels and horizontal scaling on system behavior. The results demonstrate that the proposed approach supports early detection of performance bottlenecks, reduces operational risks, and improves the reliability of software releases by incorporating performance as a continuous quality gate.

1. Introduction

DevOps is an approach that integrates software development and IT operations to improve collaboration, automate processes, and accelerate the delivery of high-quality software. In this context, practices such as Continuous Integration (CI) and Continuous Delivery (CD) have been widely adopted to automate stages such as build, testing, and deployment [3, 11].

In modern CI/CD pipelines, unit and functional tests are commonly executed automatically throughout the delivery

process. However, performance testing is often not integrated into these pipelines or is performed manually after deployment. This gap increases the risk of system degradation, latency issues, and service unavailability, especially in systems exposed to high concurrency and public access [6].

Performance testing is essential for evaluating system behavior under different workload conditions, including response time, scalability, and stability. Unlike functional testing, which focuses on correctness, performance testing assesses how systems behave under realistic operational scenarios [9].

In public-sector environments, where systems are exposed to a large number of users and may experience peak workloads, the absence of automated performance validation can lead to significant operational risks [1].

This paper proposes a framework for integrating automated performance testing into CI/CD pipelines through a parameterizable plug-in that executes load tests and records results directly within the pipeline logs. The proposed approach enables the continuous validation of non-functional requirements, such as response time, throughput, and concurrency limits, as part of the software delivery process.

The main contributions of this work are:

- An automated approach for integrating performance testing into a real CI/CD pipeline;
- A parameterizable plug-in for generating and executing load tests;
- An empirical evaluation based on a 2^2 factorial design to analyze the effects of concurrency and horizontal scaling;
- Practical insights, lessons learned, and threats to validity derived from a real-world public-sector environment.

2. Background

2.1. Continuous Integration and Continuous Delivery

Continuous Integration (CI) and Continuous Delivery (CD) are core practices in DevOps that aim to automate the software development lifecycle. In CI/CD pipelines, stages such as compilation, unit testing, static analysis, and deployment are triggered automatically based on changes in the source code repository [3, 11].

These practices improve delivery speed, reduce manual effort, and increase the reliability of software releases. However, while functional validation is commonly integrated into CI/CD pipelines, the validation of non-functional requirements, such as performance and scalability, is still not widely adopted in many organizations [10].

2.2. Performance Testing in Automated Environments

Performance testing evaluates how a system behaves under controlled environment, allowing the identification of bottlenecks, response time degradation, and scalability limitations. Unlike unit and functional tests, performance tests assess systemic characteristics that emerge from the interaction between application components, infrastructure, and workload [6, 12].

Integrating performance testing into CI/CD pipelines enables the early detection of performance issues before the system is exposed to end users. However, this integration is specially challenging since performance tests typically depend on a deployed system and may be influenced by external factors such as real user traffic and load balancing policies [1].

2.3. Factorial Experimental Design

To analyze system behavior under different workload and infrastructure conditions, a 2^2 factorial design was adopted. This experimental technique allows the evaluation of two independent factors at two levels each, as well as their interaction effects.

2.4. Related Work

Previous studies have explored performance testing and load testing methodologies in large-scale systems [6, 9]. More recent approaches have investigated the integration of testing practices within DevOps environments [1].

However, these approaches do not explicitly integrate performance validation as an automated quality gate within CI/CD pipelines. Most existing approaches treat performance testing as a separate activity or require manual execution after deployment. In contrast, the frame-

work proposed in this work integrates performance testing directly into the CI/CD pipeline as an automated stage, enabling continuous validation of non-functional requirements, which is the main contribution of this work.

Unlike prior works that rely on tools such as JMeter or manual configurations, the proposed framework emphasizes automation and seamless integration into the CI/CD lifecycle.

3. Context and Motivation

This study was conducted in a public-sector organization responsible for providing digital services to citizens and businesses. The system under analysis is based on a microservices architecture implemented in Java and deployed in a Kubernetes environment, with CI/CD pipelines automating build, testing, and deployment processes.

Although the pipeline already included automated stages for unit testing and static code analysis, performance testing was not integrated into the process. The validation of non-functional requirements, such as concurrency limits, response time, and system stability, was performed manually or only after deployment.

This scenario becomes particularly critical for services exposed to the public, which are subject to high levels of concurrent access and large volumes of data. The absence of automated performance validation increases the risk of delayed bottleneck detection, performance degradation, and potential service unavailability.

Additionally, the system architecture relies on multiple pods with load balancing and concurrency limits per instance. This configuration requires empirical validation to ensure that the system behaves as expected under different scaling conditions.

Therefore, the main motivation of this work is to reduce operational risks, enable early detection of performance issues, and incorporate the validation of non-functional requirements into the continuous delivery process through an automated and structured approach.

4. Proposed Framework

This work proposes a framework for integrating automated performance testing into CI/CD pipelines. Instead of treating performance testing as a separate activity, the framework introduces an automated post-deployment stage that enables continuous validation of non-functional requirements.

As illustrated in Figure 1, performance testing is incorporated as part of the pipeline execution, allowing early detection of performance issues and enforcement of quality gates based on predefined thresholds.



Figure 1. Overview of the proposed framework for integrating performance testing into CI/CD pipelines.

The framework consists of three main phases: (i) integration of the testing stage into the pipeline, (ii) automated execution of performance tests based on configurable workload parameters, and (iii) collection and analysis of performance metrics to support decision-making.

Based on the observed results, the pipeline can be automatically interrupted when performance requirements are not satisfied, preventing the propagation of issues to production environments.

By embedding performance validation into the CI/CD lifecycle, the framework transforms performance testing into a continuous and automated quality assurance practice.

5. Case Study

This section presents a real-world case study conducted in a public-sector organization to evaluate the applicability of the proposed framework.

5.1. System Architecture

The system under analysis is based on a microservices architecture deployed on a Kubernetes cluster [7]. Each microservice is implemented in Java using Spring Boot and exposes RESTful endpoints. Requests are routed through an API Gateway, which distributes traffic among multiple service instances (pods).

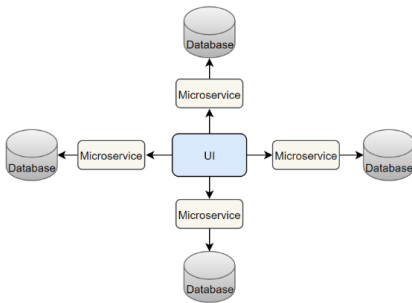


Figure 2. Basic representation of a microservices architecture.

As described in previous studies [2], each microservice represents a bounded context following Domain-Driven Design principles, promoting modularity and scalability. For this study, a critical microservice exposed to the public was selected. This service is responsible for providing large volumes of fiscal data and is subject to high levels of concurrent access. To ensure system stability, a non-functional requirement was defined: each pod should handle a maximum number of concurrent requests. Requests exceeding this limit are rejected with HTTP 503 (Service Unavailable).

5.2. Plug-in Implementation and Integration

This case study was implemented using widely adopted DevOps tools such as Jenkins [5], Docker [4], and Kubernetes [7], enabling reproducible and scalable execution of performance tests. The proposed plug-in dynamically generates test scripts in JavaScript format compatible with Grafana k6 [8], executes them within the CI/CD pipeline, and records the results directly in the logs.

The plug-in was implemented in Java to automate the execution of performance tests. It receives as input parameters the number of virtual users, test duration, and the target URLs. Based on these parameters, the plug-in dynamically generates test scripts in JavaScript format compatible with Grafana k6. After script generation, the plug-in builds the execution command, triggers the test using k6, and captures the output for logging within the CI/CD pipeline.

```
List<String> command = new
    ArrayList<>();

command.add("k6");
command.add("run");
command.add("--vus");
command.add(String.valueOf(vus));
command.add("--duration");
command.add(duration);
command.add(tempScriptPath.
    toAbsolutePath().toString());

ProcessBuilder builder = new
    ProcessBuilder(command);
builder.redirectErrorStream(true);
Process process = builder.start();
```

Figure 3. Plug-in Code Snippet

If the test execution fails or predefined thresholds are violated, the plug-in returns an error status to the pipeline, allowing the workflow to be interrupted according to the defined quality policy.

For automated execution, the plug-in was installed in Jenkins and invoked through a Jenkinsfile. The performance stage receives the test parameters and executes the plug-in as part of the pipeline execution process.

```

stage('Performance Test Execution') {
    steps {
        script {
            def urls = params.URLS.split(',')
            as List
            performanceAnalysisPlugin(
                urls: urls,
                vus: params.VUS.toInteger(),
                duration: params.DURATION
            )
        }
    }
}

```

Figure 4. Groovy script snippet for invoking the performance testing plug-in in Jenkins

5.3. Experimental Design and Execution

The evaluated microservice provides fiscal data for download by taxpayers, involving high data volume and frequent access by external users. To preserve system stability, a concurrency control mechanism was implemented per instance, limiting the number of simultaneous requests handled by each pod.

Therefore, the objective of the performance tests was to verify whether the service behaves as expected when the configured concurrency limits are exceeded, particularly observing the occurrence of HTTP 503 (Service Unavailable) responses.

Although the analysis is primarily descriptive, the factorial design provides a structured approach to understanding the influence of each factor on system behavior.

The service was deployed in a Kubernetes environment using three pods, each configured with resource requests of 200m CPU and 256MB RAM, and limits of 400m CPU and 512MB RAM. Additionally, each pod was configured to accept up to 10 simultaneous requests per IP.

The independent variables considered in the experiment were the number of pods and the concurrency level defined by the number of virtual users (VUs). The experimental scenarios were organized according to a 2^2 factorial design, as shown in Table 1.

Table 1. 2^2 factorial experimental design matrix.

Test	Concurrency (A)	Level A	Pods (B)	Level B
1	30 VUs	-1	3 Pods	-1
2	50 VUs	+1	3 Pods	-1
3	30 VUs	-1	5 Pods	+1
4	50 VUs	+1	5 Pods	+1

Each scenario was executed for ten minutes. When the workload exceeded the configured limits, the application re-

turned HTTP 503 responses, indicating that additional requests were rejected by the rate-limiting policy.

With the parameters defined, the tests were executed automatically after deployment. The execution was performed in a dedicated pipeline triggered at the end of the deployment process, and the test logs were made available directly in Jenkins.

In this study, the tests were executed in a production environment, which introduces variability due to external concurrent traffic and load balancer distribution. This aspect is further discussed in the Lessons Learned and Threats to Validity sections.

6. Results and Discussion

The figures presented in this section contain original labels in Portuguese, as they reflect the real production environment where the framework was applied.

The experimental results demonstrate that the system behaves according to the defined non-functional requirements. Under moderate concurrency levels, the number of HTTP 503 errors remained low, indicating that the rate-limiting mechanism does not interfere with normal system usage. However, when the system was subjected to higher concurrency levels, particularly in the scenario with 50 virtual users and 3 pods, a significant increase in HTTP 503 responses was observed. This behavior reflects the intended operation of the rate limit mechanism, which prevents system overload by rejecting excessive requests.

Even in scenarios theoretically within the configured system capacity, occasional HTTP 503 responses were observed. This behavior can be attributed to the non-uniform distribution of requests by the load balancer, which may temporarily concentrate requests on specific pods, causing localized overload.

The results were also organized using a 2^2 factorial design signal table, considering the number of HTTP 503 errors as the response variable, as shown in Table 2.

The factorial analysis shows that the concurrency factor has a strong impact on the number of errors, while the number of pods acts as a mitigation mechanism under high load conditions. The interaction between these factors indicates that horizontal scalability reduces the rejection rate when the system is subjected to higher levels of concurrent requests, reinforcing the importance of cluster scalability.

Another relevant aspect observed was the non-uniform request distribution by the load balancer. Even under scenarios theoretically within the total system capacity, sporadic HTTP 503 errors occurred, indicating that aggregated capacity does not always translate into uniform effective capacity under real concurrency conditions.

From an operational perspective, integrating performance testing into the CI/CD pipeline allowed transform-

Table 2. 2^2 factorial design signal table (response variable: number of HTTP 503 errors).

Concurrency (A)	Pods (B)	A	B	AB	Errors 503	Rate (%)
30 VUs	3	-1	-1	+1	1	0.02
50 VUs	3	+1	-1	-1	61	1.34
30 VUs	5	-1	+1	-1	7	0.13
50 VUs	5	+1	+1	+1	34	0.63

ing performance metrics into automated quality gates. The identification of a bottleneck related to response time illustrates this impact, as described in the following subsection.

6.1. Bottleneck Identification

In addition to validating the rate limit policy, the tests revealed a critical bottleneck related to response time. Initially, the average response time exceeded 40 seconds. Since the pipeline was configured to fail when the average response time exceeded 5 seconds, the performance stage automatically blocked the pipeline execution.

Without automated performance testing, this issue could have been detected only after the system was exposed to end users. The integration of performance testing into the pipeline enabled early detection and immediate feedback, preventing the propagation of performance issues into production environments.

After corrective actions were applied, new tests indicated a significant improvement, with average response time reduced to approximately 2 seconds even under higher concurrency levels, allowing the pipeline to complete successfully.

These results demonstrate that the proposed framework not only validates non-functional requirements but also enables the identification of performance bottlenecks and supports proactive optimization of the system before production exposure.

7. Lessons Learned

The integration of automated performance testing into a CI/CD pipeline demonstrated that technical automation alone is not sufficient to guarantee stable and reliable results if the execution environment is not properly controlled. Running performance tests directly in a production environment introduced variability caused by external concurrent requests and load balancer behavior, which can affect the interpretation of results.

It was also observed that tests based solely on virtual users do not necessarily represent a fixed request arrival rate. The latency of the application directly influences the effective throughput, which may lead to misleading conclusions about system capacity. Therefore, complementary

metrics and alternative workload models should be considered to achieve more accurate performance evaluations.

Another important observation is that horizontal scalability significantly reduces the rate of HTTP 503 errors under high concurrency conditions. However, it does not completely eliminate failures due to uneven load distribution across pods. This highlights the importance of analyzing not only theoretical system capacity but also the dynamic behavior of infrastructure components such as load balancers.

The use of a custom plug-in proved to be effective in reducing manual intervention and standardizing test execution within the pipeline. However, it introduces the need for continuous maintenance as the pipeline evolves. Additionally, test parameters must be adapted according to the characteristics of each system, as different applications have distinct performance requirements and constraints.

8. Threats to Validity

This study presents some limitations that may affect the generalization of the results. First, the performance tests were executed directly in a production environment, which introduces variability due to real user traffic and external system interactions. This may lead to false positives or false negatives when interpreting the results.

Additionally, the load balancer does not guarantee uniform distribution of requests among pods, which can result in localized overload and the occurrence of HTTP 503 errors even under theoretically acceptable load conditions. This behavior may impact the reproducibility of the experiments.

Another limitation is related to the use of virtual users as the primary load model. Since the throughput depends on response time, higher latency may reduce the effective load generated during tests, potentially underestimating the actual system capacity.

Finally, the proposed framework was evaluated in a specific organizational context and applied to a single type of microservice. Although the results are promising, further studies are necessary to validate the approach in different domains, architectures, and workload scenarios.

9. Conclusion

This paper presented a framework for integrating automated performance testing into CI/CD pipelines, enabling the continuous validation of non-functional requirements during the software delivery process. By introducing an automated stage capable of generating workloads, executing tests, and analyzing results directly within the pipeline, the proposed approach reduces manual intervention and improves the reliability of software releases.

The case study demonstrated that the framework is effective in validating concurrency limits and identifying performance bottlenecks before the system is widely exposed to end users. The use of a factorial experimental design allowed a structured analysis of the impact of concurrency and horizontal scaling, providing valuable insights for infrastructure decision-making.

In addition to validating system behavior, the integration of performance testing into the CI/CD pipeline enabled the transformation of performance metrics into objective quality gates. This approach allowed the early detection of critical issues, such as excessive response time, preventing the propagation of performance problems to production environments.

From both practical and academic perspectives, this work contributes to bridging the gap between performance engineering and modern DevOps practices [12]. It demonstrates that performance testing can evolve from an occasional activity into a continuous and automated process embedded within the software delivery lifecycle.

This work demonstrates that integrating performance testing into CI/CD pipelines is not only feasible, but essential for ensuring software reliability in modern distributed systems.

10. Future Work

Future work includes the use of ephemeral testing environments to reduce interference from external traffic and improve reproducibility of results. Additionally, future research may explore the automatic discovery of system endpoints and the integration of performance testing with service meshes and observability tools.

Another promising direction is the extension of the experimental design to include additional factors such as resource limits, workload patterns, and network latency. This would enable a more comprehensive understanding of system behavior under diverse operational conditions.

Furthermore, the application of the proposed framework in different domains, architectures, and organizational contexts may provide additional evidence of its generalizability and robustness. The integration of machine learning techniques for anomaly detection and adaptive performance

thresholds also represents a potential area for future investigation.

References

- [1] Tse-Hsun Chen, Mitchell Syer, Weiyi Shang, Zhen Ming Jiang, and Ahmed Hassan. Analytics-driven load testing. *IEEE Software*, 2017.
- [2] Melissa Tidori da Conceição and Giuliano Scombatti Pinto. Arquitetura de microsserviços. *Interface Tecnológica*, 18(2):53–64, 2021.
- [3] S. R. Dileepkumar and Juby Mathew. Optimize continuous integration and continuous deployment in azure devops for a controlled microsoft .net environment using different techniques and practices. In *IOP Conference Series: Materials Science and Engineering*, volume 1085 of *IOP Conf. Ser.: Mater. Sci. Eng.*, page 012027. IOP Publishing, 2021. Presented at AICERA 2020.
- [4] Docker. Docker platform, 2024.
- [5] Jenkins. Jenkins automation server, 2024.
- [6] Z. M. Jiang and A. E. Hassan. A survey on load testing of large-scale software systems. *IEEE Transactions on Software Engineering*, 2015.
- [7] Kubernetes. Kubernetes documentation, 2024.
- [8] Grafana Labs. k6 load testing tool, 2024.
- [9] Daniel A. Menasce. Load testing of web sites. *IEEE Internet Computing*, 2002.
- [10] Shravan Pargaonkar. A comprehensive review of performance testing methodologies and best practices: Software quality engineering. *International Journal of Science and Research (IJSR)*, 12(8):2008–2014, August 2023. Licensed under Creative Commons Attribution CC BY.
- [11] Manuel Alejandro Pastrana Pardo, Hugo Armando Ordoñez Erazo, and Carlos Alberto Cobos Lozada. Documenting and implementing devops good practices with test automation and continuous deployment tools through software refinement. *Periodicals of Engineering and Natural Sciences*, 9(4):854–863, November 2021. Licensed under Creative Commons Attribution CC BY 4.0.
- [12] M. Woodside, G. Franks, and D. C. Petriu. The future of software performance engineering. In *Future of Software Engineering*, pages 171–187. IEEE, 2007.